【超详细】【原理篇&实战篇】一文读懂Transformer

原创 ㄣ知冷煖★ 已于 2024-01-02 14:52:52 修改 阅读量4.1w 收藏 383 点赞数 95
分类专栏： 自然语言处理 深度学习 文章标签： transformer 深度学习 自然语言处理

GitCode 开源社区 文章已被社区收录

自然语言处理 同时被 2 个专栏收录 13 订阅 33 篇文章

【原理篇】一文读懂Transformer

前言
一、Transformer是什么?
1-1、 Transformer的结构:
1-2、 自注意力机制
1-3、使用位置编码表示序列的顺序
1-4、Add&Normalize
1-5、全连接层Feed Forward
1-6、Decoder整体结构
1-7、输出
1-8、transformer的优缺点:
二、Self-Attention的实现
2-0、过程
2-1、准备输入（词嵌入向量）
2-2、初始化参数（Q、K、V矩阵）
2-3、获取key，query和value
2-4、计算注意力分数
2-5、计算softmax
2-6、给value乘上score
2-7、给value加权求和获取output(得到input1的结果向量)
三、Transformer代码实现
总结

前言

Transformer是一种用于自然语言处理（NLP）和其他序列到序列（sequence-to-sequence）任务的深度学习模型架构，它在2017年由Vaswani等人首
Transformer架构引入了自注意力机制（self-attention mechanism），这是一个关键的创新，使其在处理序列数据时表现出色。



一、Transformer是什么?

Transformer是一种用于自然语言处理 （NLP）和其他序列到序列（sequence-to-sequence）任务的深度学习模型架构，它在2017年由Vaswani提出。Transformer架构引入了自注意力机制（self-attention mechanism），这是一个关键的创新，使其在处理序列数据时表现出色。
以下是Transformer的一些重要组成部分和特点：

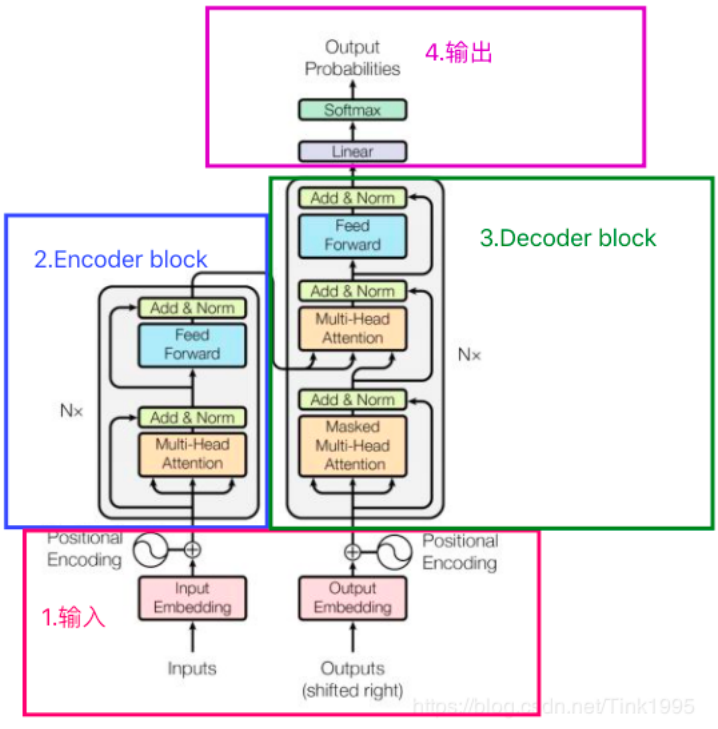
- 自注意力机制（Self-Attention）：这是Transformer的核心概念之一，它使模型能够同时考虑输入序列中的所有位置，而不是像循环神经网络（RNN）或卷积神经（CNN）一样逐步处理。自注意力机制允许模型根据输入序列中的不同部分来赋予不同的注意权重，从而更好地捕捉语义关系。
- 多头注意力（Multi-Head Attention）：Transformer中的自注意力机制被扩展为多个注意力头，每个头可以学习不同的注意权重，以更好地捕捉不同类型的关系。允许模型并行处理不同的信息子空间。
- 堆叠层（Stacked Layers）：Transformer通常由多个相同的编码器和解码器层堆叠而成。这些堆叠的层有助于模型学习复杂的特征表示和语义。
- 位置编码（Positional Encoding）：由于Transformer没有内置的序列位置信息，它需要额外的位置编码来表达输入序列中单词的位置顺序。
- 残差连接和层归一化（Residual Connections and Layer Normalization）：这些技术有助于减轻训练过程中的梯度消失和爆炸问题，使模型更容易训练。
- 编码器和解码器：Transformer通常包括一个编码器用于处理输入序列和一个解码器用于生成输出序列，这使其适用于序列到序列的任务，如机器翻译。



CSDN @L,知冷煖★

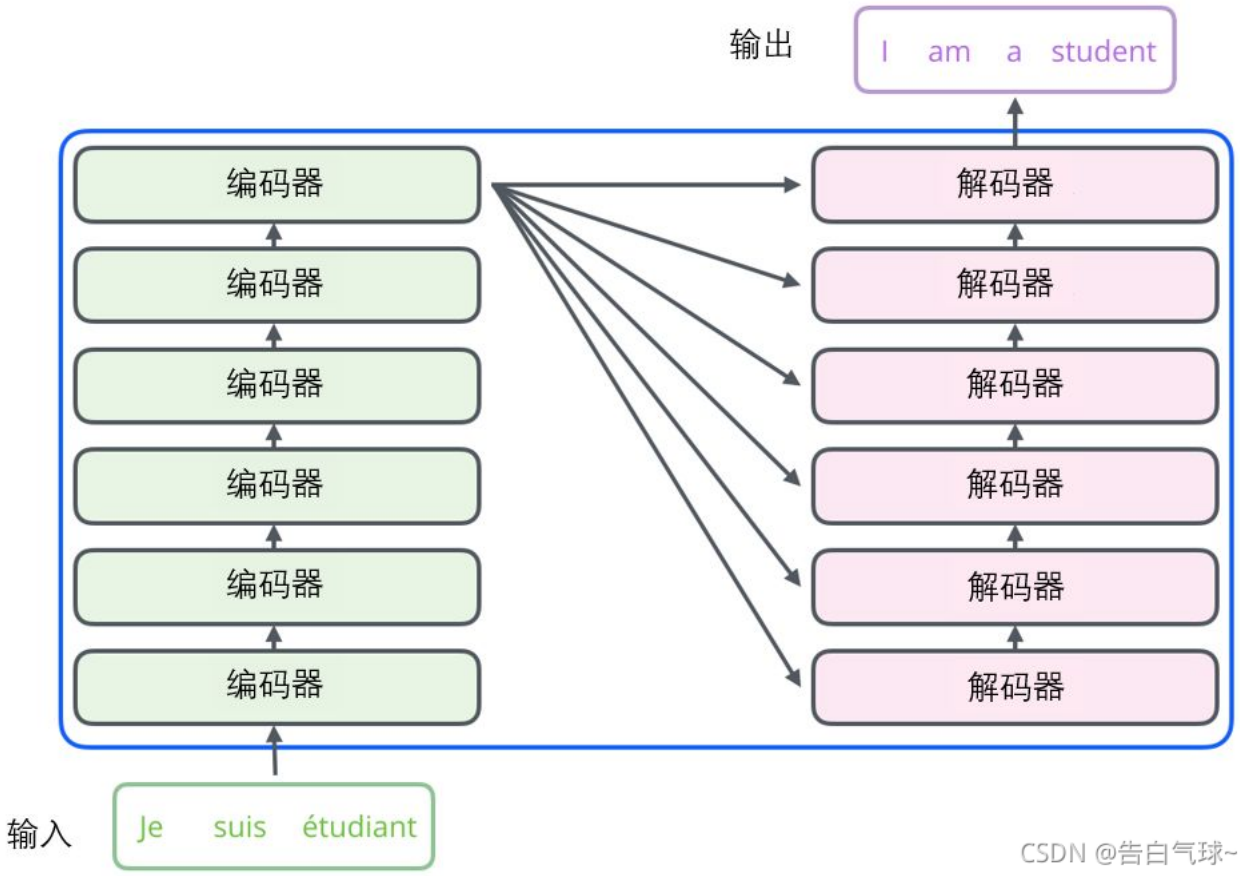
1-1、 Transformer的结构：

Nx = 6，Encoder block由6个encoder堆叠而成，图中的一个框代表的是一个encoder的内部结构，一个Encoder是由Multi-Head Attention和全连接Feed Forward Network构成。如下图所示：

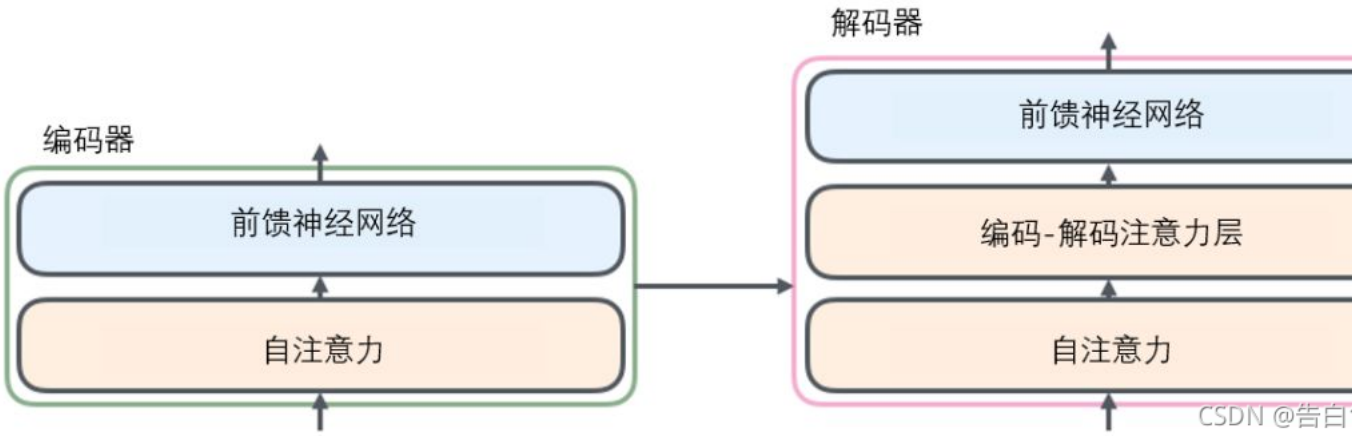


https://blog.csdn.net/Tink1995
CSDN @告白气球~

简略结构（每一个编码器都对应上图的一个encoder结构）：



Transformer的编码组件是由6个编码器叠加在一起组成的，解码器同样如此。所有的编码器在结构上是相同的，但是它们之间并没有共享参数。编码
构如下：



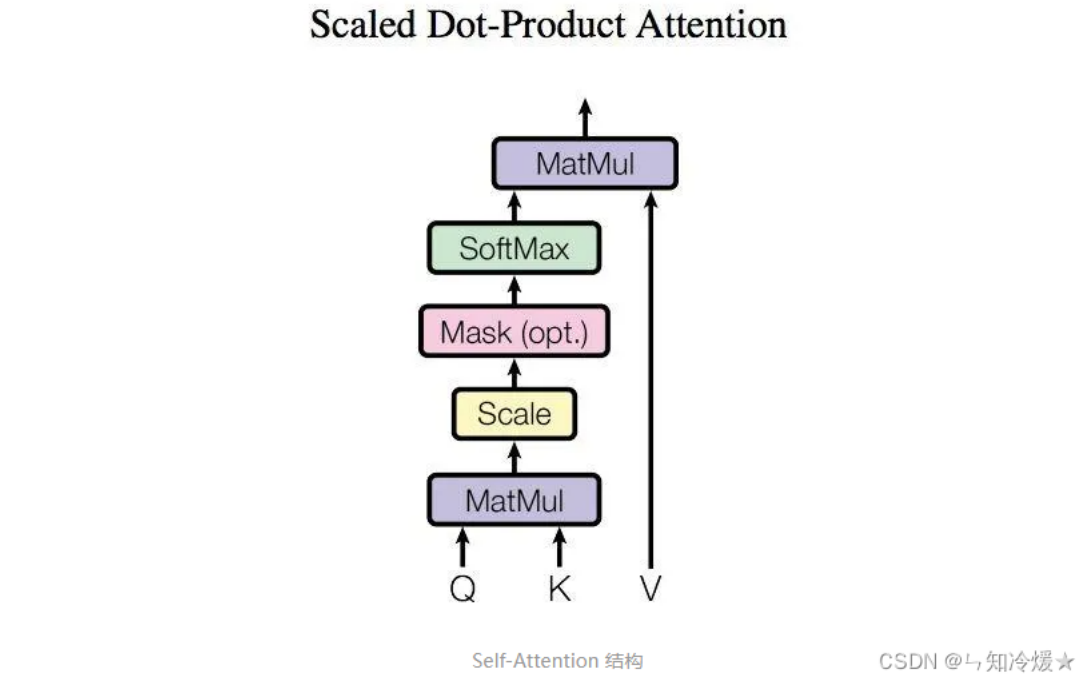
从编码器输入的句子首先会经过一个自注意力层，这一层帮助编码器在对每个单词编码的时候时刻关注句子的其它单词。解码器中的解码注意力层的
输入句子的相关部分，类似于seq2seq的注意力。原结构中使用到的是多头注意力机制（Multi-Head Attention），我们先从基础——自注意力机制开

1-2、 自注意力机制

自注意力的作用：随着模型处理输入序列的每个单词，自注意力会关注整个输入序列的所有单词，帮助模型对本单词更好地进行编码。在处理过程中
机制会将对所有相关单词的理解融入到我们正在处理的单词中。更具体的功能如下：

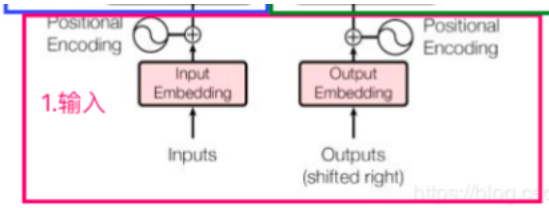
- 序列建模：自注意力可以用于序列数据（例如文本、时间序列、音频等）的建模。它可以捕捉序列中不同位置的依赖关系，从而更好地理解上下
  机器翻译、文本生成、情感分析等任务非常有用。
- 并行计算：自注意力可以并行计算，这意味着可以有效地在现代硬件上进行加速。相比于RNN和CNN等序列模型，它更容易在GPU和TPU等硬件
  的训练和推理。（**因为在自注意力中可以并行的计算得分**）
- 长距离依赖捕捉：传统的循环神经网络（RNN）在处理长序列时可能面临梯度消失或梯度爆炸的问题。自注意力可以更好地处理长距离依赖关系
  需要按顺序处理输入序列。

自注意力的结构如下所示：



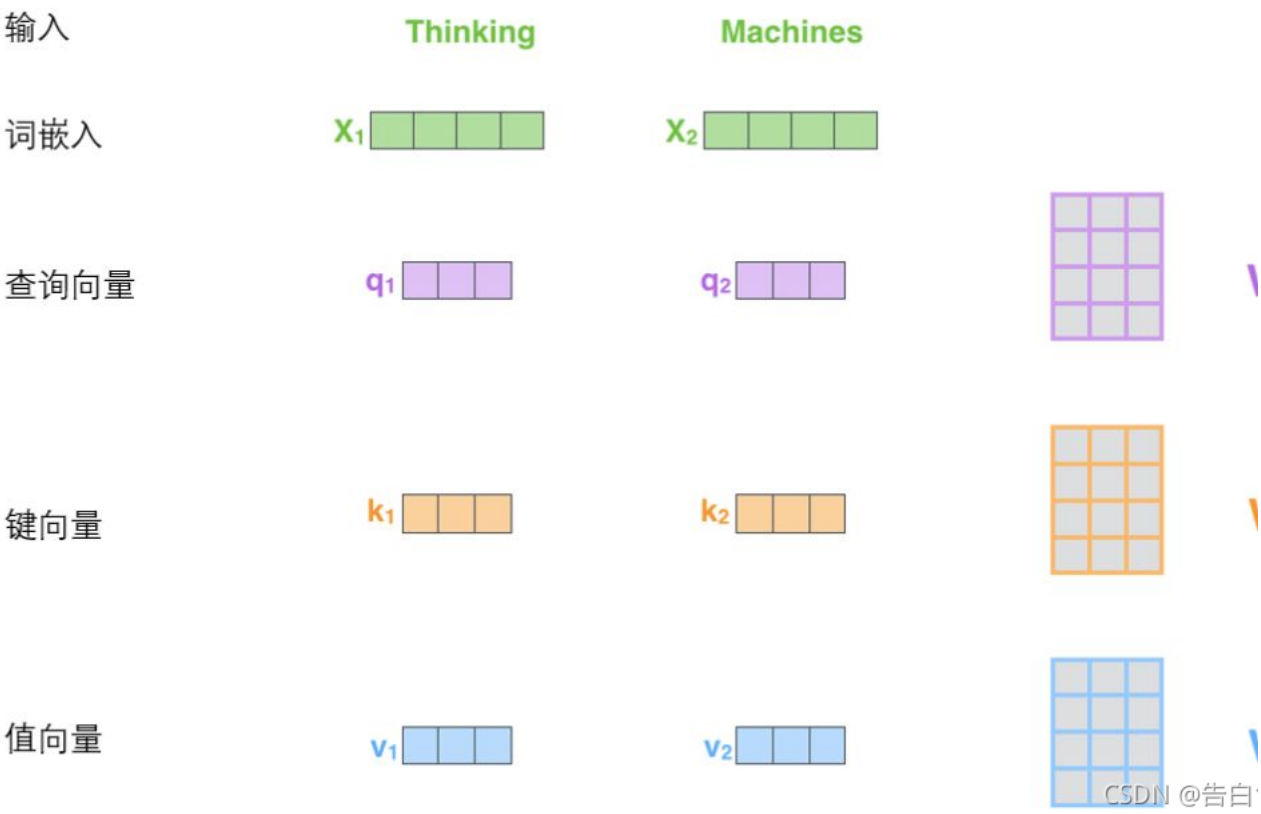
Self-Attention 结构　　CSDN @ㄣ知冷煖★

自注意力的计算：从每个编码器的输入向量（每个单词的词向量，即Embedding，可以是任意形式的词向量，比如说word2vec，GloVe，one-hot编码



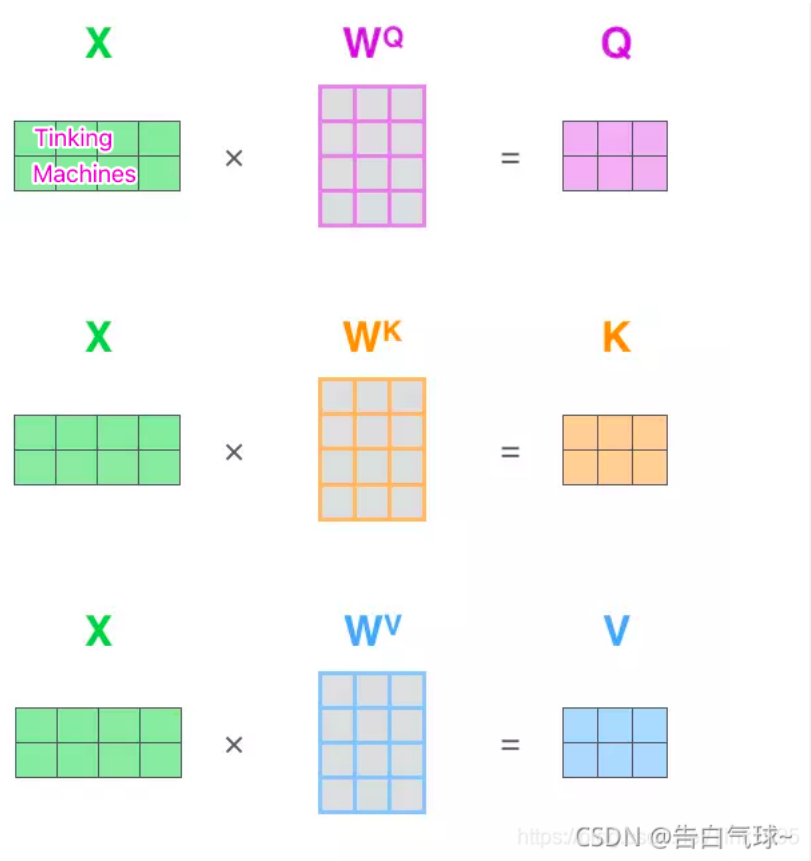
CSDN @ㄣ知冷煖★

中生成三个向量，即**查询向量、键向量和一个值向量。**（这三个向量是通过词嵌入与三个权重矩阵即W^Q, W^K, W^V,相乘后创建出来的）新向量在维比词嵌入向量更低。（512->64）



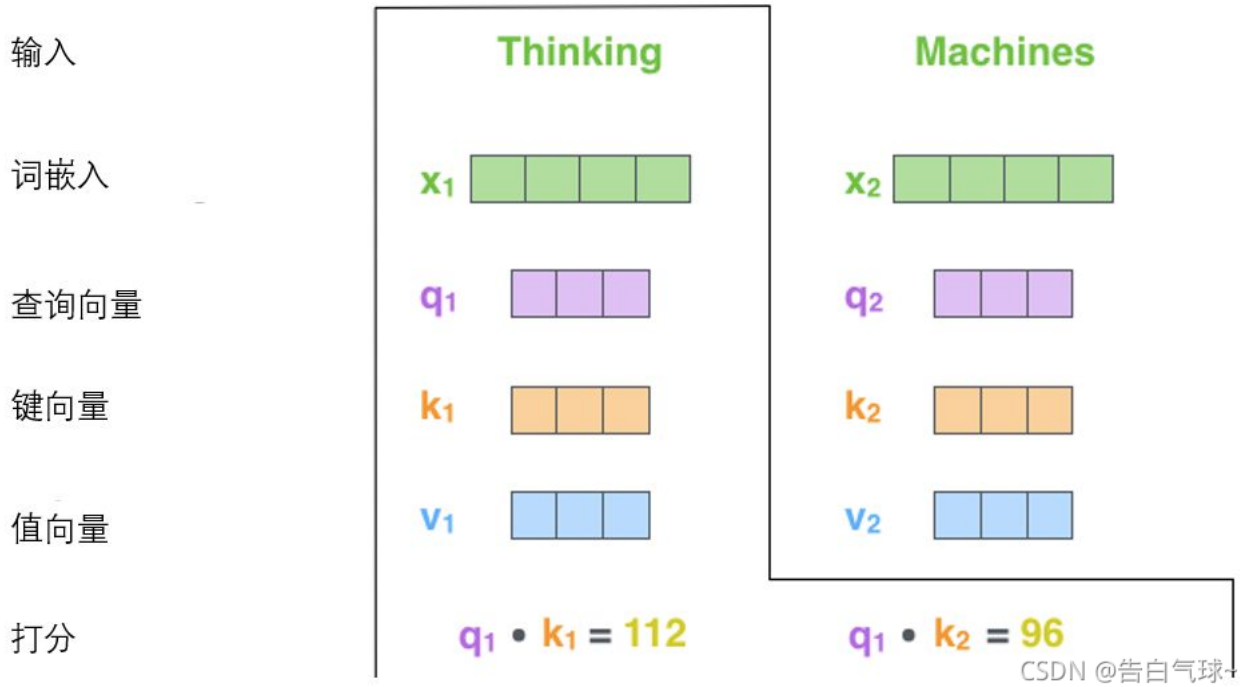
CSDN @告白

更一般的，将以上所得到的查询向量、键向量、值向量组合起来就可以得到三个向量矩阵Query、Keys、Values。

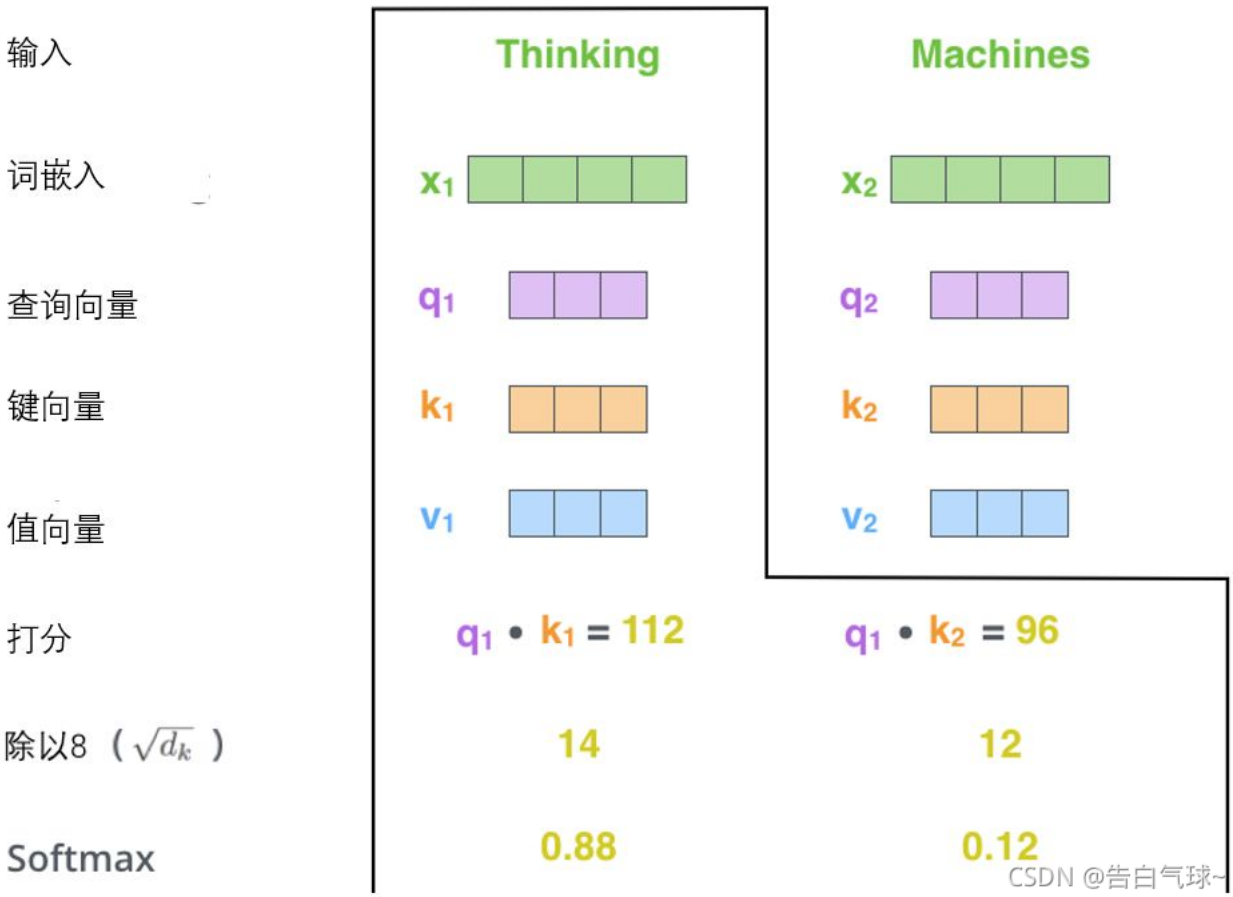


查询向量、键向量和值向量

计算自注意力的**第二步**是计算得分，假设我们在为这个例子中的第一个词“Thinking”计算自注意力向量，我们需要拿输入句子中的每个单词对“Thinking
些分数是通过所有输入句子的单词的键向量与“Thinking”的查询向量相点积来计算的。



第三步和第四步是将分数除以8(8是论文中使用的键向量的维数64的平方根，这会让梯度更稳定。这里也可以使用其它值，8只是默认值，这样做是为
过大。)，然后通过softmax传递结果。随着模型处理输入序列的每个单词，自注意力会关注整个输入序列的所有单词，帮助模型对本单词更好地进行
softmax的作用是使所有单词的分数归一化，得到的分数都是正值且和为1。

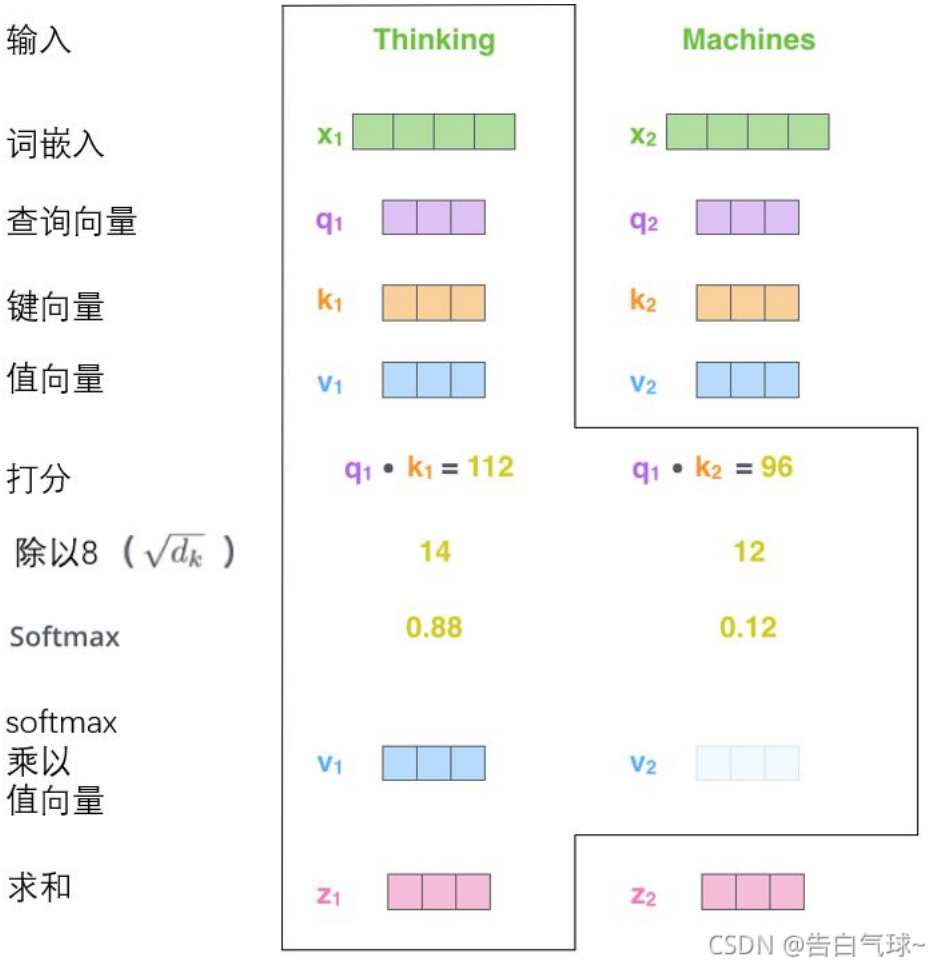


这个softmax分数决定了每个单词对编码当下位置（“Thinking”）的贡献。显然，已经在这个位置上的单词将获得最高的softmax分数，

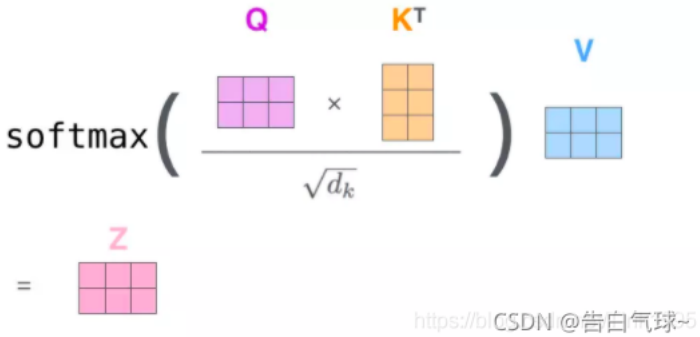
第五步是将每个值向量乘以softmax分数(这是为了准备之后将它们求和)。这里的直觉是希望关注语义上相关的单词，并弱化不相关的单词(例如，让它0.001这样的小数)。

Softmax函数：或称归一化指数函数，它将每一个元素的范围都压缩到（0，1）之间，并且所有元素的和为1。

第六步是对加权值向量求和，然后即得到自注意力层在该位置的输出(在我们的例子中是对于第一个单词)。



整体的计算图如图所示：



最终得到了自注意力，并将得到的向量传递给前馈神经网络。以上二到六步合为一个公式计算自注意力层的输出。

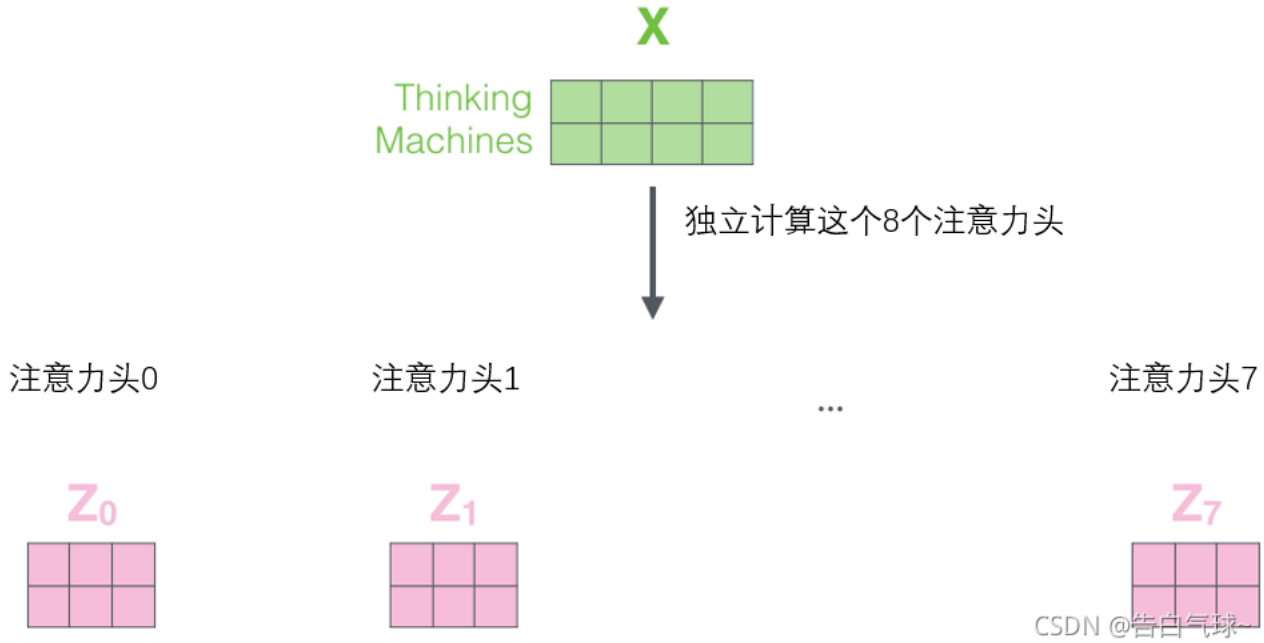
自注意力层的完善——“多头”注意力机制：

对应整体结构图中的**Multi——Head Attention**

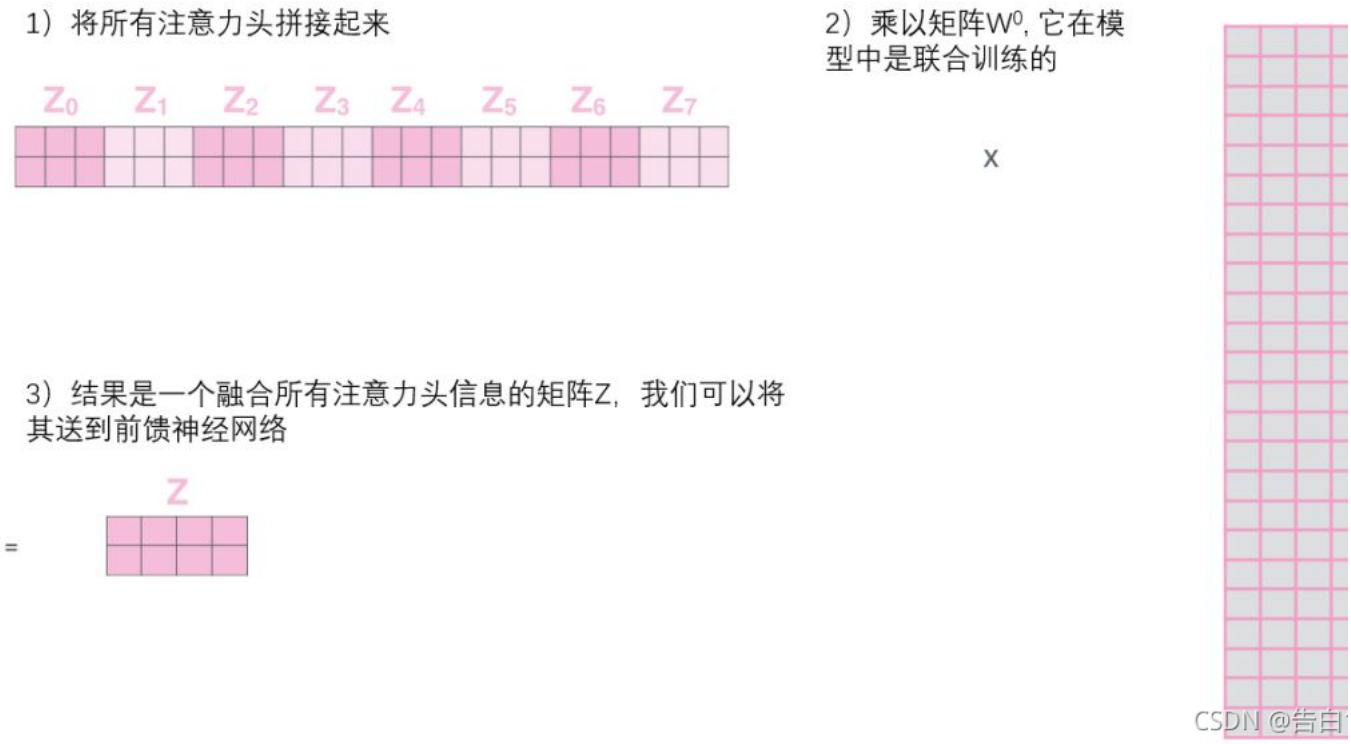
1、扩展了模型专注于不同位置的能力。

2、有多个查询/键/值权重矩阵集合，（Transformer使用八个注意力头）并且每一个都是随机初始化的。和上边一样，用矩阵X乘以WQ、WK、WV来键、值矩阵。

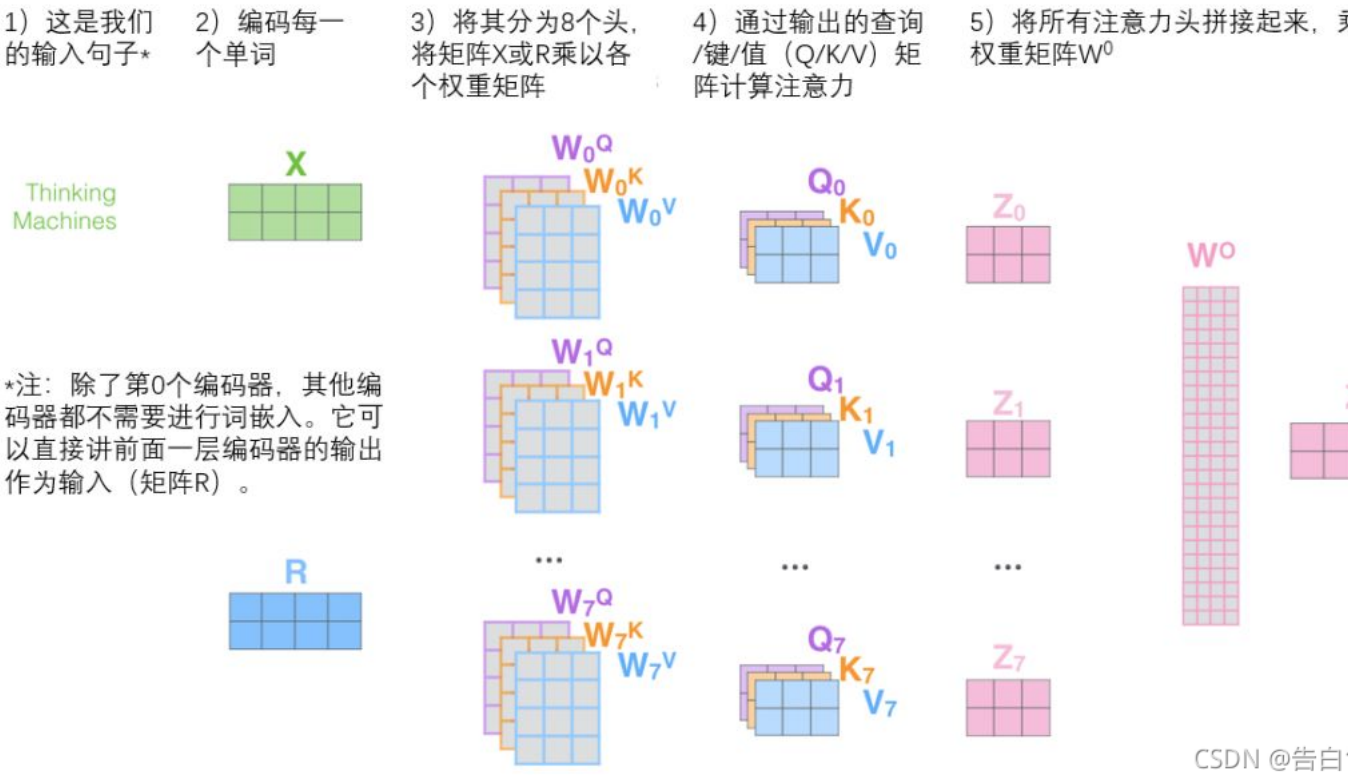
3、**self-attention**只是使用了一组WQ、WK、WV来进行变换得到查询、键、值矩阵，而Multi-Head Attention使用多组WQ，WK，WV得到多组查询、阵，然后每组分别计算得到一个Z矩阵。



前馈层只需要一个矩阵，则把得到的8个矩阵拼接在一起，然后用一个附加的权重矩阵W^O与它们相乘。

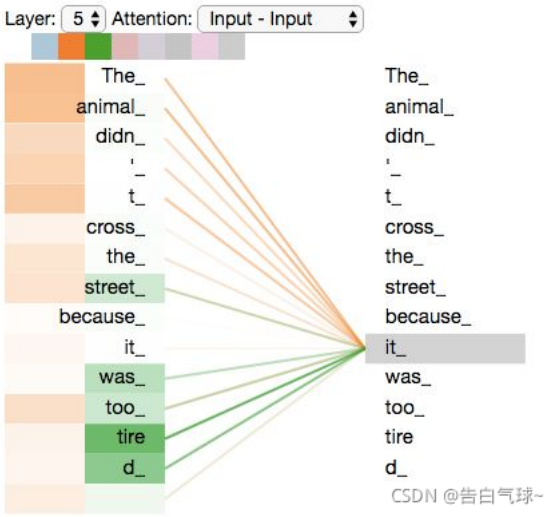


总结整个流程：



编码it一词时，不同注意力的头集中在哪里，当我们编码it这个单词时：（图中只列举出了两个注意力）：

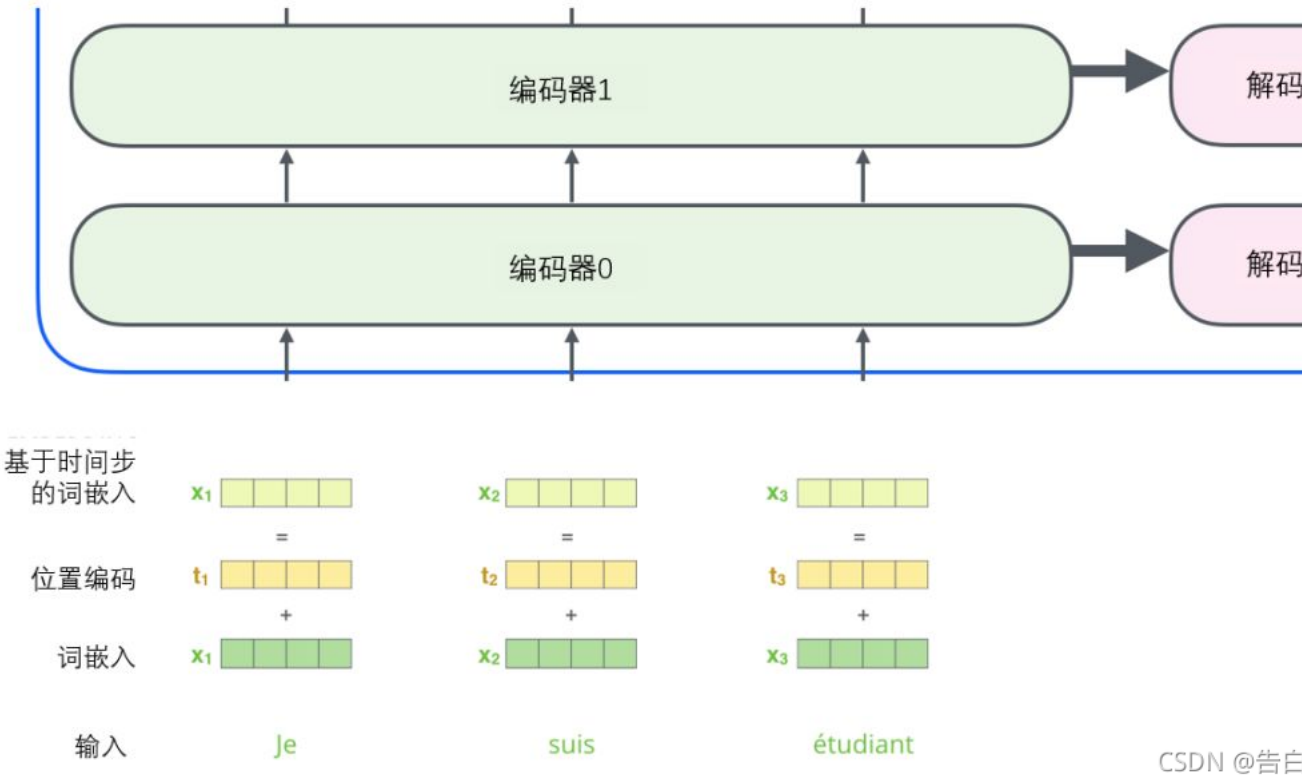
- 其中一个注意力头集中在The animal
- 另一个注意力头集中在tire上。即形象解释it代指的是animal和tire。



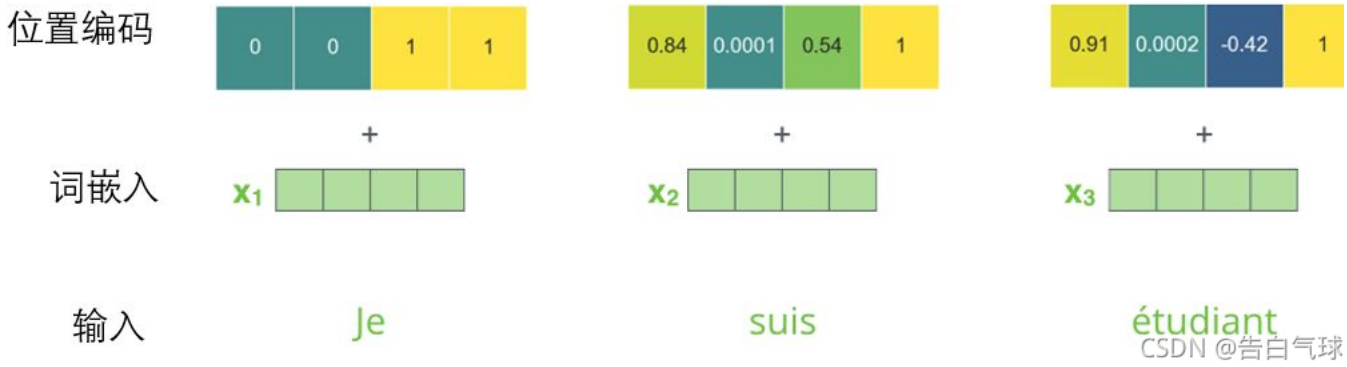
1-3、使用位置编码表示序列的顺序

为什么要用位置编码?

- 如果不添加位置编码，那么无论单词在什么位置，它的注意力分数都是确定的。这不是我们想要的。
- 为了理解单词顺序，Transformer为每个输入的词嵌入添加了一个向量，这样能够更好的表达词与词之间的关系。词嵌入与位置编码相加，而不是
的效率差不多，但是拼接的话维度会变大，所以不考虑。（这里位置向量如何得到，以哪种计算方法得到，以及词嵌入与位置编码如何结合是可
的点，可以看以下链接思考这个问题）。如何理解Transformer论文中的positional encoding，和三角函数有什么关系？.



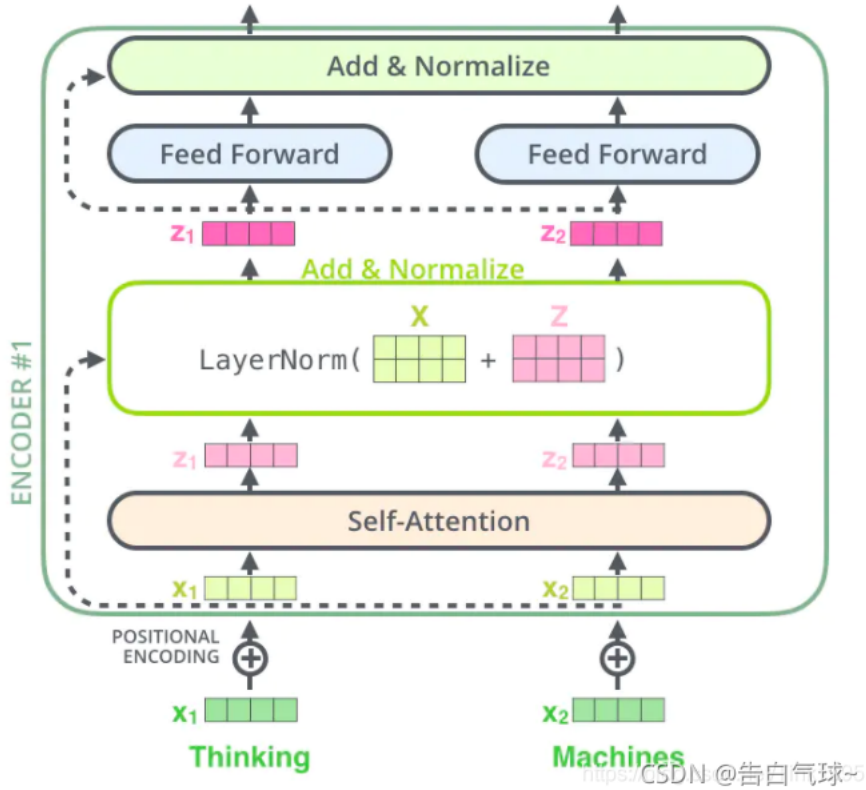
为了让模型理解单词的顺序，我们添加了位置编码向量，这些向量的值遵循特定的模式。



CSDN @告白气球

1-4、Add&Normalize

在经过多头注意力机制得到矩阵Z之后，并没有直接传入全连接神经网络，而是经过了一步Add&Normalize。



CSDN @告白气球~

Add & Norm 层由 Add 和 Norm 两部分组成，其计算公式如下：

$$\text{LayerNorm}\big(X + \text{MultiHeadAttention}(X)\big)$$
$$\text{LayerNorm}\big(X + \text{FeedForward}(X)\big)$$

CSDN @★_寒冷煖★

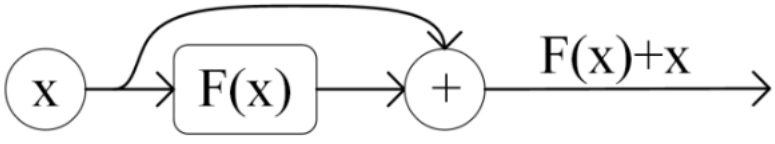
其中 X表示 Multi-Head Attention 或者 Feed Forward 的输入，MultiHeadAttention(X) 和 FeedForward(X) 表示输出 (输出与输入 X 维度是一样的，所以

加)。

- **Add**
  Add，就是在z的基础上加了一个残差块X，加入残差块的目的是为了防止在深度神经网络的训练过程中发生退化的问题，退化的意思就是深度神经

  增加网络的层数，Loss逐渐减小，然后趋于稳定达到饱和，然后再继续增加网络层数，Loss反而增大。

Add指 X+MultiHeadAttention(X)，是一种残差连接，通常用于解决多层网络训练的问题，可以让网络只关注当前差异的部分，在 ResNet 中经常用到：



残差连接

CSDN @ㄣ知冷煖★

- **ResNet残差神经网络**

  为了了解残差块，我们引入ResNet残差神经网络，神经网络退化指的是在达到最优网络层数之后，神经网络还在继续训练导致Loss增大，对于多
  们需要保证多出来的网络进行恒等映射。只有进行了恒等映射之后才能保证这多出来的神经网络不会影响到模型的效果。**残差连接主要是为了防
  化。**

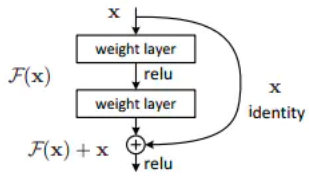


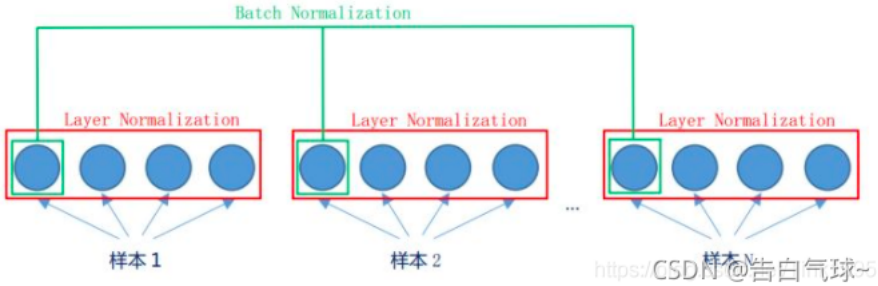
Figure 2. Residual learning: a building block.

  上图就是构造的一个残差块，X是输入值，F（X）是经过第一层线性变换后并且激活的输出，在第二层线性变化之后，激活之前，F（X）加入了
  值X，然后再进行激活后输出。

要恒等映射，我们只需要让F（X）=0就可以了。x经过线性变换（随机初始化权重一般偏向于0），输出值明显会偏向于0，而且经过激活函数Relu会
0，过滤了负数的影响。
这样当网络自己决定哪些网络层为冗余层时，使用ResNet的网络很大程度上解决了学习恒等映射的问题，用学习残差F(x)=0更新该冗余层的参数来代
h(x)=x更新冗余层的参数。

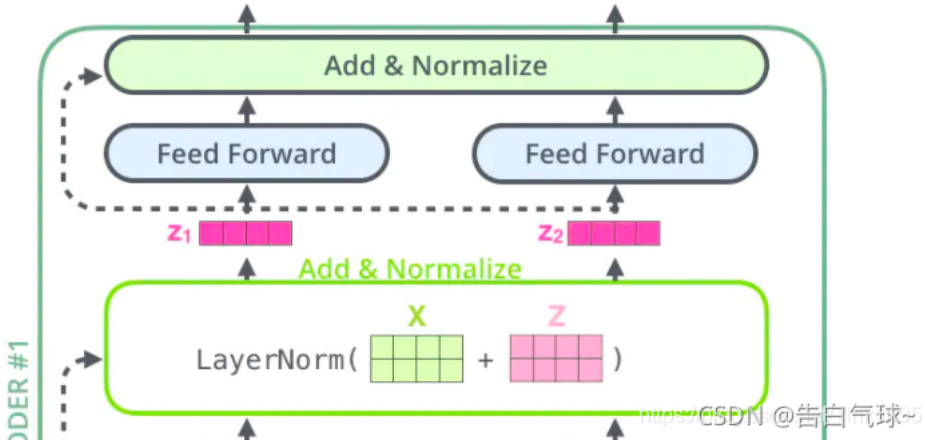
- **Normalize**

  归一化目的：
  1、加快训练速度
  2、提高训练的稳定性
  使用到的归一化方法是Layer Normalization。



CSDN @告白气球~

  LN是在同一个样本中不同神经元之间进行归一化，而BN是在同一个batch中不同样本之间的同一位置的神经元之间进行归一化。
  BN是对于相同的维度进行归一化，但是咱们NLP中输入的都是词向量，一个300维的词向量，单独去分析它的每一维是没有意义地，在每一维上
  也是适合地，因此这里选用的是LN。



CSDN @告白气球~

1-5、全连接层Feed Forward

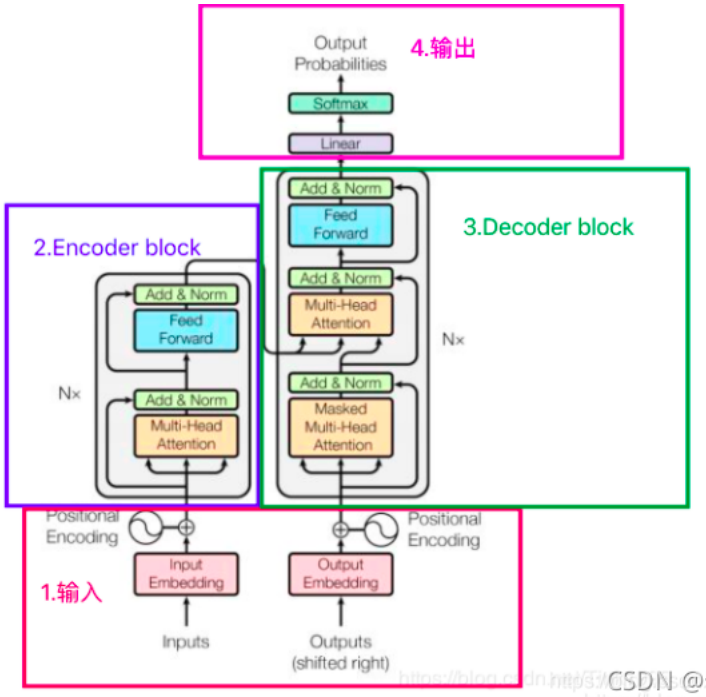
全连接层公式如下：

$$\mathrm{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

CSDN @ㄣ知冷煖★

全连接层是一个两层的神经网络，先线性变换，然后ReLU非线性，再线性变换。
这两层网络就是为了将输入的Z映射到更加高维的空间中然后通过非线性函数ReLU进行筛选，筛选完后再变回原来的维度。
经过6个encoder后输入到decoder中。

1-6、Decoder整体结构



CSDN @告白气球~

和Encoder Block一样，Encoder也是由6个decoder堆叠而成的，Nx=6。包含两个 Multi-Head Attention 层。第一个 Multi-Head Attention 层采用操作。第二个 Multi-Head Attention 层的K, V矩阵使用 Encoder 的编码信息矩阵C进行计算，而Q使用上一个 Decoder block 的输出计算。

Masked Multi-Head Attention
与Encoder的Multi-Head Attention计算原理一样，只是多加了一个mask码。mask 表示掩码，它对某些值进行掩盖，使其在参数更新时不产生效果。T模型里面涉及两种 mask，分别是 padding mask 和 sequence mask。为什么需要添加这两种mask码呢?

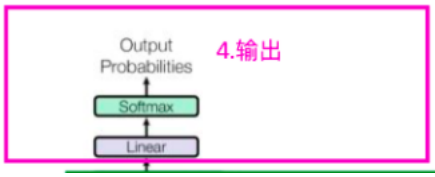
- **padding mask**
  什么是 padding mask 呢？因为每个批次输入序列长度是不一样的也就是说，我们要对输入序列进行对齐。具体来说，就是给在较短的序列后面填是如果输入的序列太长，则是截取左边的内容，把多余的直接舍弃。因为这些填充的位置，其实是没什么意义的，所以我们的attention机制不应该放在这些位置上，所以我们需要进行一些处理。
  具体的做法是，把这些位置的值加上一个非常大的负数(负无穷)，这样的话，经过 softmax，这些位置的概率就会接近0！

- **sequence mask**
  sequence mask 是为了使得 decoder 不能看见未来的信息。对于一个序列，在 time_step 为 t 的时刻，我们的解码输出应该只能依赖于 t 时刻之前而不能依赖 t 之后的输出。因此我们需要想一个办法，把 t 之后的信息给隐藏起来。这在训练的时候有效，因为训练的时候每次我们是将target数据进decoder中地，预测时不需要，预测的时候我们只能得到前一时刻预测出的输出。
  那么具体怎么做呢？也很简单：产生一个上三角矩阵，上三角的值全为0。把这个矩阵作用在每一个序列上，就可以达到我们的目的。

注意：
1、在Encoder中的Multi-Head Attention也是需要进行mask的，只不过Encoder中只需要padding mask即可，而Decoder中需要padding mask和mask。
2、Encoder中的Multi-Head Attention是基于Self-Attention地，Decoder中的第二个Multi-Head Attention就只是基于Attention，它的输入Quer来Masked Multi-Head Attention的输出，Keys和Values来自于Encoder中最后一层的输出。

1-7、输出

Output如图中所示，首先经过一次线性变换（线性变换层是一个简单的全连接神经网络，它可以把解码组件产生的向量投射到一个比它大得多的，被称率的向量里），然后Softmax得到输出的概率分布（softmax层会把向量变成概率），然后通过词典，输出概率最大的对应的单词作为我们的预测输出



1-8、transformer的优缺点：

优点：
1、效果好
2、可以并行训练，速度快
3、很好的解决了长距离依赖的问题
缺点：
完全基于self-attention，对于词语位置之间的信息有一定的丢失，虽然加入了positional encoding来解决这个问题，但也还存在着可以优化的地方。

二、Self-Attention的实现

2-0、过程

- 准备输入
- 初始化参数
- 获取key，query和value
- 给input1计算attention score
- 计算softmax
- 给value乘上score
- 给value加权求和获取output1
- 重复步骤4-7，获取output2，output3

2-1、准备输入（词嵌入向量）

```
import torch
x = [
  [1, 0, 1, 0], # Input 1
  [0, 2, 0, 2], # Input 2
  [1, 1, 1, 1]  # Input 3
 ]
x = torch.tensor(x, dtype=torch.float32)
x
```

输出：
tensor([[1., 0., 1., 0.],
[0., 2., 0., 2.],
[1., 1., 1., 1.]])

2-2、初始化参数（Q、K、V矩阵）

Note： Q、K、V矩阵在神经网络初始化的过程中，一般都是随机采样完成并且比较小，可以根据想要输出的维度来确定 Q、K、V矩阵的维度。

```
w_key = [
  [0, 0, 1],
  [1, 1, 0],
  [0, 1, 0],
  [1, 1, 0]
]
w_query = [
  [1, 0, 1],
  [1, 0, 0],
  [0, 0, 1],
  [0, 1, 1]
]
w_value = [
  [0, 2, 0],
  [0, 3, 0],

```

```
  [1, 0, 3],
  [1, 1, 0]
]
w_key = torch.tensor(w_key, dtype=torch.float32)
w_query = torch.tensor(w_query, dtype=torch.float32)
w_value = torch.tensor(w_value, dtype=torch.float32)

print("Weights for key: \n", w_key)
print("Weights for query: \n", w_query)
print("Weights for value: \n", w_value)
```

输出:

Weights for key:
tensor([[0., 0., 1.],
[1., 1., 0.],
[0., 1., 0.],
[1., 1., 0.]])
Weights for query:
tensor([[1., 0., 1.],
[1., 0., 0.],
[0., 0., 1.],
[0., 1., 1.]])
Weights for value:
tensor([[0., 2., 0.],
[0., 3., 0.],
[1., 0., 3.],
[1., 1., 0.]])

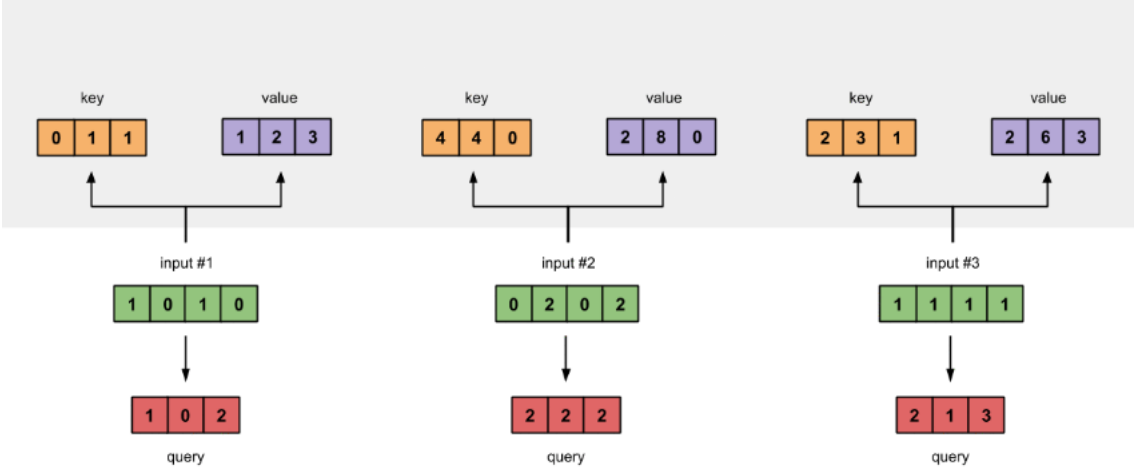
2-3、获取key，query和value

```
keys = x @ w_key
querys = x @ w_query
values = x @ w_value

print("Keys: \n", keys)
# tensor([[0., 1., 1.],
#         [4., 4., 0.],
#         [2., 3., 1.]])

print("Querys: \n", querys)
# tensor([[1., 0., 2.],
#         [2., 2., 2.],
#         [2., 1., 3.]])
print("Values: \n", values)
# tensor([[1., 2., 3.],
#         [2., 8., 0.],
#         [2., 6., 3.]])
```

下图为得到的key，query和value：



CSDN @ㄣ知冷煖★

2-4、计算注意力分数

```
attn_scores = querys @ keys.T
print(attn_scores)
```

输出：

tensor([[2., 4., 4.],
[4., 16., 12.],
[4., 12., 10.]])

2-5、计算softmax

```
from torch.nn.functional import softmax

attn_scores_softmax = softmax(attn_scores, dim=-1)
print(attn_scores_softmax)
# tensor([[6.3379e-02, 4.6831e-01, 4.6831e-01],
#         [6.0337e-06, 9.8201e-01, 1.7986e-02],
#         [2.9539e-04, 8.8054e-01, 1.1917e-01]])

# 为了使得后续方便，这里简略将计算后得到的分数赋予了一个新的值
# For readability, approximate the above as follows
attn_scores_softmax = [
  [0.0, 0.5, 0.5],
  [0.0, 1.0, 0.0],
  [0.0, 0.9, 0.1]
]
attn_scores_softmax = torch.tensor(attn_scores_softmax)
print(attn_scores_softmax)
```

输出：

tensor([[6.3379e-02, 4.6831e-01, 4.6831e-01],
[6.0337e-06, 9.8201e-01, 1.7986e-02],
[2.9539e-04, 8.8054e-01, 1.1917e-01]])
tensor([[0.0000, 0.5000, 0.5000],
[0.0000, 1.0000, 0.0000],
[0.0000, 0.9000, 0.1000]])

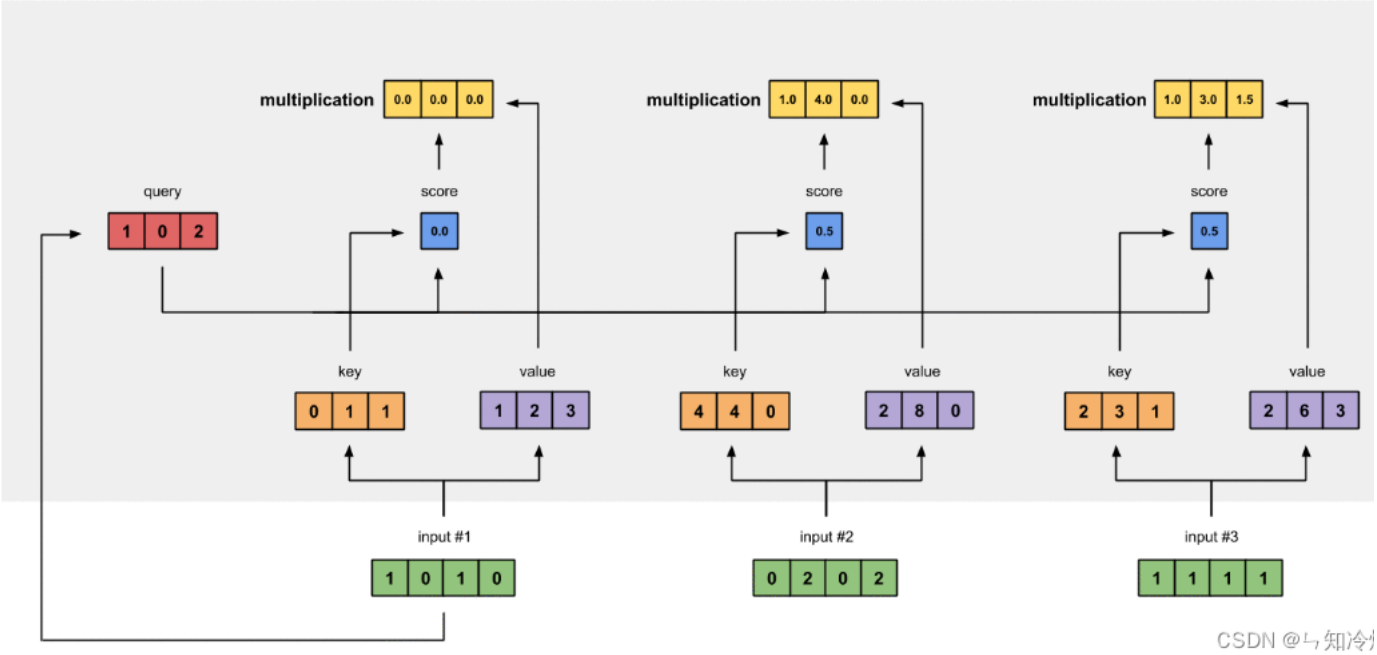
2-6、给value乘上score

```
weighted_values = values[:,None] * attn_scores_softmax.T[:,:,None]
print(weighted_values)
```

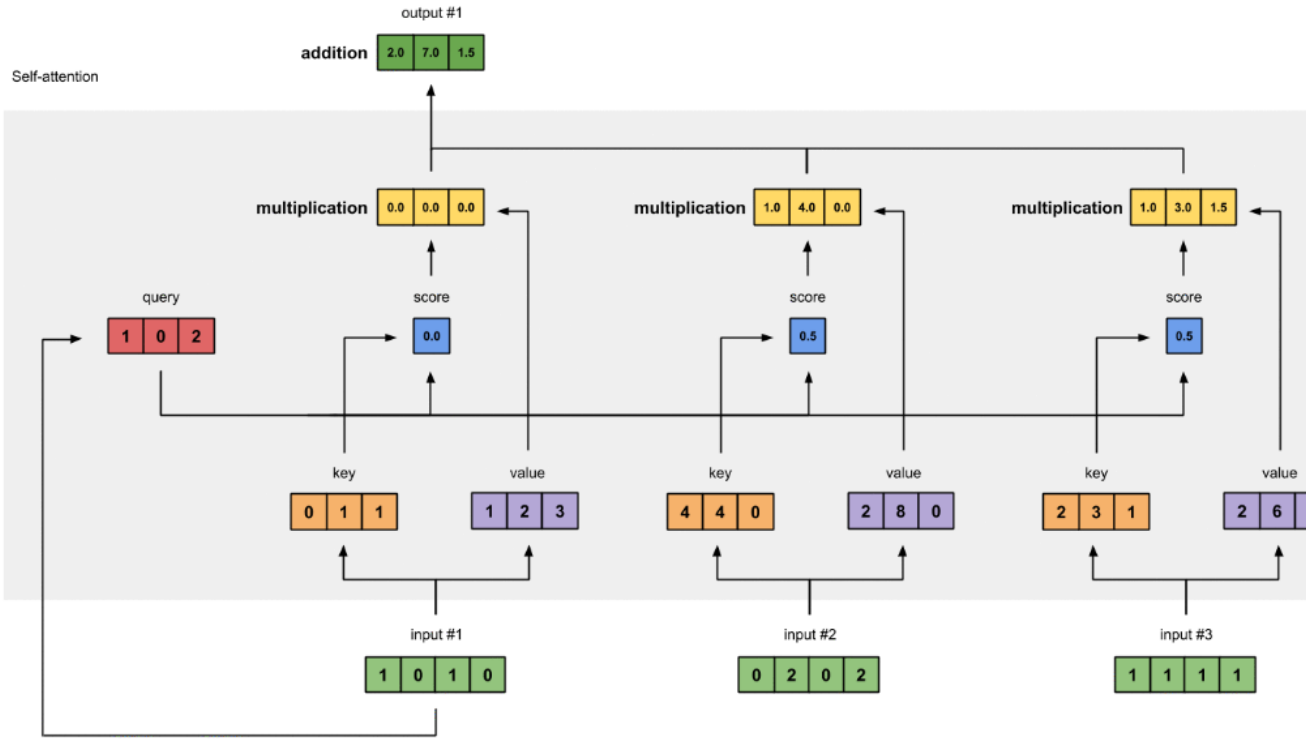
输出：

tensor([[[0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000],

[0.0000, 0.0000, 0.0000]],
[[1.0000, 4.0000, 0.0000],
[2.0000, 8.0000, 0.0000],
[1.8000, 7.2000, 0.0000]],
[[1.0000, 3.0000, 1.5000],
[0.0000, 0.0000, 0.0000],
[0.2000, 0.6000, 0.3000]]])



2-7、给value加权求和获取output(得到input1的结果向量)



把所有的weighted values（黄色）进行element-wise的相加。

```
  [0.0, 0.0, 0.0]
+ [1.0, 4.0, 0.0]
+ [1.0, 3.0, 1.5]
-----------------
= [2.0, 7.0, 1.5]
```

得到结果向量[2.0, 7.0, 1.5]（深绿色）就是ouput1的和其他key交互的query representation。

CSDN @L

重复步骤4-7，获取到input2、input3的结果向量

三、Transformer代码实现

```
1
```

参考文章:

图解Transformer（完整版）.
史上最小白之Transformer详解.
一文读懂BERT(原理篇).
Transformer模型详解（图解最完整版）.
【从 0 开始学习 Transformer】 上篇：Transformer 搭建与理解.
Transformer 一篇就够了（一）： Self-attenstion.
TRANSFORMERS FROM SCRATCH.
【Transformer】10分钟学会Transformer | Pytorch代码讲解 | 代码可运行.
Transformer 模型详解.
三万字最全解析！从零实现Transformer（小白必会版）.
算法工程师当前选哪个方向好？.

总结

以上就是我总结（照抄）其它优秀博主的全部transformer内容了。

ㄣ知冷煖★
微信名片

显

Transformer原理详解和代码_transformer代码实现
Transformer是一种Encoder-Decoder结构的NLP模型,在模型通过self-attention机制代替了循环神经网络和卷积网络,有效缓解了长序列首尾信息难以交互和序列模型无法并

Transformer模型详解
2. Transformer 的输入 Transformer 中单词的输入表示x由单词 Embedding和位置 Embedding(Positional Encoding)相加得到。 Transformer 的输入表示 2.1. 单词 Embedd

机器学习算法那些事 | 使用Transformer模型进行时间序列预测实战 **最新发布** csdn_xmj的
时间序列预测是一个经久不衰的主题，受自然语言处理领域的成功启发，transformer模型也在时间序列预测有了很大的发展。本文可以作为学习使用Transformer 模型的时

Transformer详解 qq_52302919的
近期Transformer MLP系列模型的出现，增加了CV领域的多样性。但是Transformer这一不同领域的模型对学习者来说需要一个细致的学习过程.下面就是本菜鸟总结学习路

【Transformer系列】深入浅出理解Transformer网络模型(综合篇)_tran...
Transformer 中除了Embedding(词嵌入),还需要使用Positional Embedding表示单词出现在句子中的位置。因为 Transformer 不采用 RNN 的结构,而是使用全局信息,不能利

Transformer详解
Transformer详解 1. 简介 Transformer是一个面向sequence to sequence任务的模型,在17年的论文《Attention is all you need》中首次提出。Transformer 是第一个完全依

深入了解Transformer模型及其优缺点 cooldream2009的
在当今人工智能领域，自然语言处理的关键问题之一是解决文本理解和生成中的挑战。传统的循环神经网络虽然在处理序列数据方面取得了一定成就，但它们存在着顺序

【itransformer 多变量时间序列 】 weixin_44064434的
*输入数据：**多变量时间序列。

【学习笔记】Transformer详解_transformer csdn
【学习笔记】Transformer详解 参考链接:【手撕Transformer】Transformer输入输出细节以及代码实现(pytorch),Transformer模型详解(图解最完整版) 模型结构 1、结构图

【Transformer系列(4)】Transformer模型结构超详细解读_transformer结构...
【Transformer系列(3)】《Attention Is All You Need》论文超详细解读(翻译+精读) 目录 前言 一、Transformer 整体结构 二、编码器:Encoder 2.1输入 2.1.1词嵌入:W

变形金刚——Transformer入门刨析详解 m0_67505927的
Transformer详解

Transformer是什么？看完这篇你就醍醐灌顶 fs1341825137的
前言 由谷歌团队提出的预训练语言模型BERT近年来正在各大自然语言处理任务中屠榜（话说学者们也挺有意思的，模型名都强行凑个芝麻街的人物名，哈哈哈）。 BER

你真的看懂Transformer了吗?小白学习笔记为你答疑解惑
1.Transformer概述 Transformer是一种基于注意力机制的序列到序列模型,它在机器翻译任务中表现出色并逐渐成为自然语言处理领域的主流模型。Transformer模型的核心

【计算机视觉 | Transformer】魔改Transformer!9种提速又提效的模型优...
1.1 Longformer: The Long-Document Transformer 1.2 Enhancing the Locality and Breaking the Memory Bottleneck of Transformer on Time Series Forecasting 1.3 Adapt

你真的看懂Transformer了吗？小白学习笔记为你答疑解惑 xuleoo的
Transformer是一种基于注意力机制的序列到序列模型，它在机器翻译任务中表现出色并逐渐成为自然语言处理领域的主流模型。Transformer模型的核心思想是使用自注意

Transformer 模型详解 **热门推荐** 步入人工智
本内容主要介绍 Transformer 模型的具体实现。

深入了解Transformer模型及其优缺点_transformer的特点
并行计算能力:相比传统的循环神经网络(RNN)模型,Transformer模型中多头注意力机制的并行计算极大提高了训练和推理的效率,允许更大规模的模型和更长的序列处理。

Transformer学习总结——原理篇
首先从整体上看一下Transformer的结构：从图中可以看出，整体上Transformer由四部分组成：Inputs:Inputs=WordEmbedding(Inputs)+PositionalEmbeddingInputs=Word

深度学习-Transformer实战系列
深度学习-Transformer实战系列课程，视频+源码+数据+文档

深度学习-Transformer实战系列课程
深度学习-Transformer实战系列视频教程，2022新课，目前市面上为数不多的关于Transformer的课程，希望对大家的学习有所帮助

基于Transformer模型的智能问答原理详解
图一就是Transformer模型的框架，不过这里的encoder和decoder不再是RNN结构，拆开来看，细节如图二：原始论文里，作者设置了6层encoder与6层decoder结构。至

深度学习 Transformer架构解析 mengxianglong123的
2018年10月，Google发出一篇论文《BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding》, BERT模型横空出世, 并横扫NLP领域11项任

BERT大火却不懂Transformer？读这一篇就够了 BigDataDigest的

大数据文摘与百度NLP联合出品编译：张驰、毅航、Conrad、龙心尘编者按：前一段时间谷歌推出的BERT模型在11项NLP任务中夺得STOA结果，引爆了整个NLP界。而

超详细Transformer 讲解 qq_40550715的博

这篇文章分为以下几个部分 Transformer 直观认识 Positional Encoding Self Attention Mechanism 残差连接和 Layer Normalization Transformer Encoder 整体结构 Transfo

transformer

Transformer是一种用于自然语言处理和机器翻译等任务的深度学习模型。它是由Google提出的，并在2017年的论文《Attention Is All You Need》中首次介绍。Transforme